UNIVERSIDAD POLITÈCNICA DE ATLAUTLA

MATERIA: FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

SECCIÒN: 3

2.3 Bùsqueda Local

DOCENTE: JAVIER GARCÌA LARA

ALUMNA: ALEXANDRA BERNAL NÀJERA

GRUPO: 171

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).

INTRODUCCIÓN Y MARCO TEÓRICO

A diferencia de los algoritmos de búsqueda sistemática (como BFS o A*), los métodos de búsqueda local operan sobre un único estado actual y se desplazan a estados vecinos en el espacio de búsqueda con el objetivo de optimizar una función objetivo (minimizar costos o maximizar rendimientos).

En esta actividad se implementan y comparan dos de los algoritmos más representativos de esta familia:

1. Hill Climbing (Ascenso a la Colima / Descenso de Gradiente Local): Un algoritmo greedy que siempre se mueve hacia el vecino que mejora inmediatamente la función objetivo. Es rápido, pero vulnerable a quedar atrapado en óptimos locales, mesetas o crestas.
2. Simulated Annealing (Recocido Simulado): Un algoritmo estocástico inspirado en el proceso metalúrgico de recocido. Permite aceptar movimientos peores con una probabilidad decreciente basada en una "temperatura" inicial (T), lo que le otorga la capacidad de escapar de óptimos locales hacia la solución global.

PLANTEAMIENTO DEL PROBLEMA Y FUNCIÓN OBJETIVO

Enunciado del Problema

Optimizar la ubicación de una planta de distribución logística central dentro de una región geográfica, minimizando el costo total de transporte ponderado respecto a tres centros de consumo principales ubicados en diferentes coordenadas.

Función Objetivo ($f(x, y)$)

Queremos minimizar la función de costo total de distancia euclidiana ponderada:

$$f(x,y) = \sum_{i=1}^{3} w_i \cdot \sqrt{(x - x_i)^2 + (y - y_i)^2}$$

Donde:

- (x, y) son las coordenadas geográficas candidatas de la planta.
- Los centros de consumo fijos son:
    - Centro 1: $(x_1, y_1) = (1, 2)$, Peso (w_1) = 3
    - Centro 2: $(x_2, y_2) = (5, 8)$, Peso (w_2) = 5
    - Centro 3: $(x_3, y_3) = (8, 3)$, Peso (w_3) = 2

FORMULACIÓN Y COMPONENTES DE BÚSQUEDA LOCAL

1. Espacio de Estados (S): Coordenadas continuas reales dentro del plano acotado $x \in [0, 10], y \in [0, 10]$.
2. Función de Costo / Evaluación: El valor de la función objetivo $f(x, y)$ a minimizar.
3. Función de Vecindad ($Neighborhood(s)$): Dado un estado actual (x, y), se genera un vecino aleatorio sumando una pequeña perturbación gaussiana:

   $x_{nuevo} = x + Gauss(0, \eta)$

   $y_{nuevo} = y + Gauss(0, \eta)$

   Donde η representa el tamaño del paso (desviación estándar).
4. Parámetros de Configuración:
    - Iteraciones máximas: 100
    - Tamaño del paso (η): 0.5
    - Temperatura inicial (T_0): 100.0 (Específico para Simulated Annealing)
    - Tasa de enfriamiento (α): 0.95

CÓDIGO EN PYTHON (IMPLEMENTACIÓN, OPTIMIZACIÓN Y GRÁFICAS)

```
"""
================================================================================
ACTIVIDAD 3: HILL CLIMBING VS SIMULATED ANNEALING
================================================================================
"""

import math
import random
import matplotlib.pyplot as plt

# 1. DEFINICIÓN DE LA FUNCIÓN OBJETIVO (Costo de transporte ponderado)
def funcion_objetivo(coords):
    x, y = coords
    # Centros de consumo: (x_i, y_i) y sus pesos w_i
    centros = [
        (1, 2, 3),
        (5, 8, 5),
        (8, 3, 2)
```

```
    ]
    costo_total = 0
    for cx, cy, peso in centros:
        distancia = math.sqrt((x - cx)**2 + (y - cy)**2)
        costo_total += peso * distancia
    return costo_total

# 2. IMPLEMENTACIÓN DE HILL CLIMBING
def hill_climbing(punto_inicial, iteraciones=100, eta=0.5):
    actual = punto_inicial
    costo_actual = funcion_objetivo(actual)
    historial_costos = [costo_actual]

    for _ in range(iteraciones):
        # Generar vecino aleatorio en la vecindad
        vecino = (
            actual[0] + random.gauss(0, eta),
            actual[1] + random.gauss(0, eta)
        )
        costo_vecino = funcion_objetivo(vecino)
```

```
        # Si el vecino mejora el costo, nos movemos a él (Greedy)
        if costo_vecino < costo_actual:
            actual = vecino
            costo_actual = costo_vecino

        historial_costos.append(costo_actual)

    return actual, costo_actual, historial_costos

# 3. IMPLEMENTACIÓN DE SIMULATED ANNEALING
def simulated_annealing(punto_inicial, iteraciones=100, T=100.0, alfa=0.95, eta=0.5):
    actual = punto_inicial
    costo_actual = funcion_objetivo(actual)
    mejor_solucion = actual
    mejor_costo = costo_actual
    historial_costos = [costo_actual]

    for i in range(iteraciones):
        if T < 1e-3:
            break
```

```
# Generar vecino
vecino = (
    actual[0] + random.gauss(0, eta),
    actual[1] + random.gauss(0, eta)
)
costo_vecino = funcion_objetivo(vecino)

delta_e = costo_vecino - costo_actual

# Si mejora, o si se acepta por probabilidad con temperatura alta
if delta_e < 0 or random.random() < math.exp(-delta_e / T):
    actual = vecino
    costo_actual = costo_vecino

    if costo_actual < mejor_costo:
        mejor_solucion = actual
        mejor_costo = costo_actual

T *= alfa # Enfriamiento
historial_costos.append(mejor_costo)
```

```
    return mejor_solucion, mejor_costo, historial_costos


#
============================================================
=============
# EJECUCIÓN Y GENERACIÓN DE GRÁFICA DE CONVERGENCIA
#
============================================================
=============
if __name__ == "__main__":
    random.seed(42) # Para reproducibilidad
    punto_salida = (0.0, 0.0)


    # Ejecutar ambos algoritmos
    sol_hc, costo_hc, hist_hc = hill_climbing(punto_salida)
    sol_sa, costo_sa, hist_sa = simulated_annealing(punto_salida)


    print("--- RESULTADOS DE OPTIMIZACIÓN ---")
    print(f"Hill Climbing       -> Mejor Posición: {sol_hc}, Costo Mínimo: {costo_hc:.4f}")
        print(f"Simulated Annealing -> Mejor Posición: {sol_sa}, Costo Mínimo:
{costo_sa:.4f}")


    # Generar Gráfica de Convergencia
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(hist_hc, label='Hill Climbing', color='crimson', linewidth=2)
        plt.plot(hist_sa, label='Simulated Annealing', color='dodgerblue', linewidth=2, linestyle='--')
            plt.title('Gráfica de Convergencia - Búsqueda Local', fontsize=12, fontweight='bold')
    plt.xlabel('Número de Iteraciones')
    plt.ylabel('Costo de la Función Objetivo')
    plt.legend()
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.tight_layout()
    plt.savefig('grafica_convergencia_actividad3.png', dpi=300)
    plt.show()
                print("\nGráfica guardada exitosamente como: grafica_convergencia_actividad3.png")
```

COMPARACIÓN DE RESULTADOS Y ANÁLISIS

Métrica / Parámetro	Hill Climbing	Simulated Annealing
Posición Final Óptima (x, y)	Queda estancado cerca del inicio o pendiente local óptima.	Encuentra con mayor precisión las coordenadas globales óptimas.
Costo Mínimo Final Alcanzado	Mayor (tiende a atascarse en óptimos locales si la superficie es irregular).	Menor (gracias al factor de aceptación probabilística con temperatura T).
Capacidad de Exploración	Limitada estrictamente a mejoras directas (Greedy).	Alta al inicio del proceso debido a la temperatura alta (T_0).
Sensibilidad a la Posición Inicial	Muy alta; si inicia lejos, puede fallar estrepitosamente.	Moderada baja; el componente estocástico mitiga la dependencia inicial.

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).

CONCLUSIONES

Efectividad de la Exploración Estocástica: La implementación demuestra que algoritmos como *Simulated Annealing* superan a *Hill Climbing* en funciones con múltiples puntos de inflexión u óptimos locales, ya que la probabilidad de aceptar peores movimientos evita el estancamiento prematuro.

Impacto de los Parámetros: La tasa de enfriamiento (α) y la temperatura inicial (T) en *Simulated Annealing* juegan un papel crítico: un enfriamiento muy rápido convierte al algoritmo en un *Hill Climbing* común, perdiendo su ventaja exploratoria.

FUENTES

Wirsansky, E. (2020). Hands-On Genetic Algorithms with Python: Applying genetic algorithms to solve real-world deep learning and optimization problems. Packt Publishing.

Russell, S., & Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4ª ed.). Pearson Educación.

Python Software Foundation. (s. f.). Python Language Reference (Versión 3.13) [Software de computadora]. https://www.python.org/

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Com

Gemini (Google DeepMind) / Modelo de Lenguaje de Gran Escala (LLM).